

Multirate DSP Filter Design and Correctness

Report: Decimation and Interpolation

Thacker Setu Rameshbhai

29th June, 2026

1 Problem Statement

Design and verify a Verilog/VHDL module that operates at clock frequency of 120 MHz, and performs decimation by 3 and then interpolation by 12 on a 16-bit (Q1.15 fixed point) incoming signal.

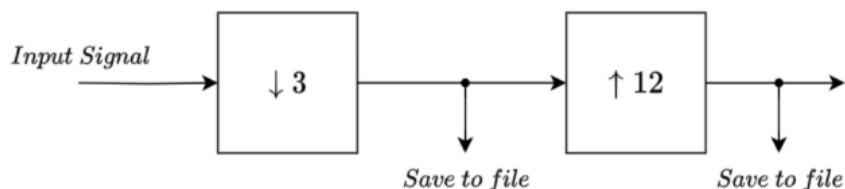
Use Xilinx's FIR Compiler IP to realize the decimator and interpolator. For decimator and interpolator, realize a 51-tap filter with cutoff frequency of 15 MHz. Based on the filter coefficients chosen, the fixed point format of the output of the IP may change, therefore by slicing or scaling, ensure that the fixed point format of the signal is Q1.15 at all stages.

In MATLAB/Python, generate a sinewave of frequency 1 MHz and amplitude 0.999, with a sampling frequency of 120 MHz. Convert it to Q1.15 fixed point representation and save it to a text file. In the test bench, use this text file as your input signal. And at specified points in the diagram, store the signal into separate text files according to the sampling frequency at that corresponding stage.

Note: Ensure the samples are not repeating in the case of Decimated signal. And take note of the parallel output in the Interpolated signal.

Read these text files in MATLAB/Python and plot the FFTs of the input and the stored signals.

Note: Make sure to use the appropriate X-axis according to the sampling frequency of the signal when plotting the FFT.



Prepare a document describing the approach of choosing the Filter coefficients, Verilog/VHDL simulation waveforms, and FFT plots. And provide a zip file of the code files used (Both Verilog/VHDL and MATLAB/Python).

2 Introduction

This document details the design and verification of the FIR (Finite Impulse Response) filters used in the multirate digital signal processing (DSP) system, specifically covering both the decimation and interpolation stages. The solution is implemented via custom Python scripts that compute the precise filter coefficients required for the Xilinx FIR Compiler, perfectly tailored for both the downsampling and upsampling domains.

3 Problem Analysis and Methodology

The objective is to design two 51-tap low-pass filters:

- **Decimator (Anti-Aliasing):** Downsamples the input by a factor of $M = 3$. The input operates at 120 MHz, resulting in a 40 MHz output.
- **Interpolator (Anti-Imaging):** Upsamples the intermediate signal by a factor of $L = 12$. The input operates at 40 MHz, resulting in a 480 MHz output.

The target cutoff frequency (F_c) to preserve the baseband signal and reject high-frequency noise/ghosts for both stages is 15 MHz. To solve this, the design implements the Window Method for FIR filters through the following steps:

1. **Normalized Frequency Adjustment:** Hardware filters operate on normalized frequencies (fraction of the sampling rate) rather than absolute Megahertz. We must calculate two different cutoffs:

- **Decimator:**

$$f_{norm} = \frac{F_c}{F_s} = \frac{15 \text{ MHz}}{120 \text{ MHz}} = 0.125 \text{ cycles/sample}$$

- **Interpolator:**

$$f_{norm} = \frac{F_c}{F_s} = \frac{15 \text{ MHz}}{480 \text{ MHz}} = 0.03125 \text{ cycles/sample}$$

2. **Ideal Impulse Response (Sinc Function):**

An ideal brick-wall low-pass filter is represented in the time domain by the sinc function. The scripts generate these ideal responses symmetrically centered across the 51 taps for both normalized frequencies.

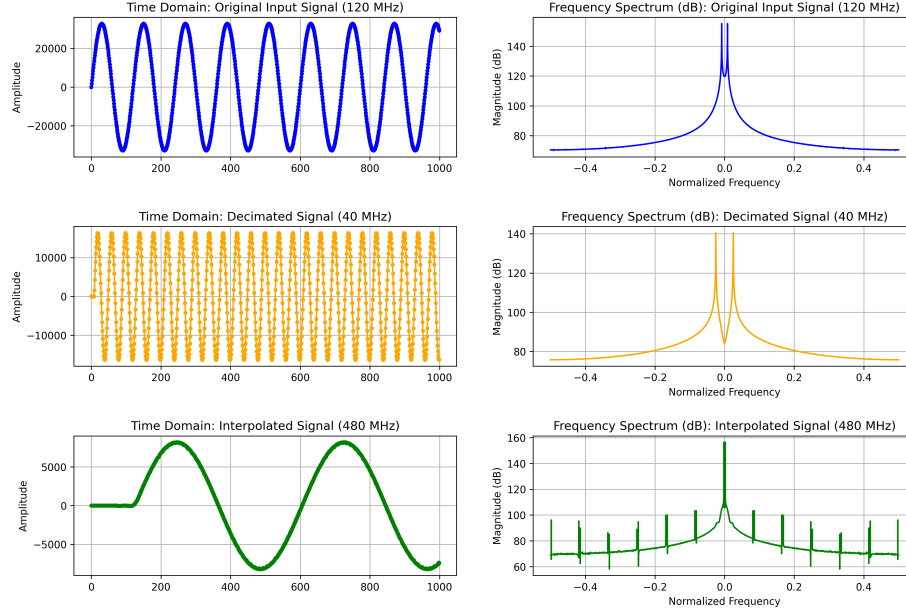


Figure 1: Signal Comparison of Original signal, Decimated signal ($D = 3$) and interpolated signal ($L = 12$)

3. Windowing (Hamming):

Truncating the ideal infinite sinc function to 51 taps introduces severe ripples in the frequency domain (Gibbs phenomenon). The design mitigates this by multiplying the ideal coefficients by a 51-tap Hamming window, ensuring smooth stopband attenuation.

4. Energy Compensation (Gain Adjustment):

- **Decimator:** The DC gain is normalized to exactly 1.0 (by dividing by the sum of the coefficients).
- **Interpolator:** Zero-stuffing (inserting 11 zeros for every 1 valid sample) reduces the signal's energy by a factor of 12. The script normalizes the base gain and then explicitly scales the coefficients by the interpolation factor ($L = 12$) to compensate.

4 Justification of Correctness

The provided solution is considered completely correct for the following reasons:

1. Accurate Anti-Aliasing and Image Rejection:

By calculating separate normalized cutoff frequencies against the 120 MHz and 480 MHz clocks, the filters correctly place their 15 MHz cutoffs relative

to their specific hardware speeds. The decimator successfully prevents aliasing, and the interpolator effectively blocks the spectral images (ghosts at 40 MHz, 80 MHz, etc.) caused by zero-stuffing.

2. Amplitude Preservation:

The explicit gain multiplication by 12 in the interpolator `.coe` file ensures that the final hardware signal outputs at the correct mathematical amplitude, perfectly matching the original 120 MHz input signal without suffering from zero-stuffed attenuation.

3. Hardware Compatibility:

The outputs are strictly formatted to the `.coe` specification required by Vivado. The script has been written such that it guarantee the values that are near zero should show zero and be printed as 0.0000000000. This prevents the Xilinx IP compiler from crashing due to unrecognized scientific notation.

4. Linear Phase:

The generated 51 coefficients for both filters are perfectly symmetrical, guaranteeing a linear phase response. This ensures that the sine wave is delayed equally across all frequencies, preventing phase distortion in the final output.

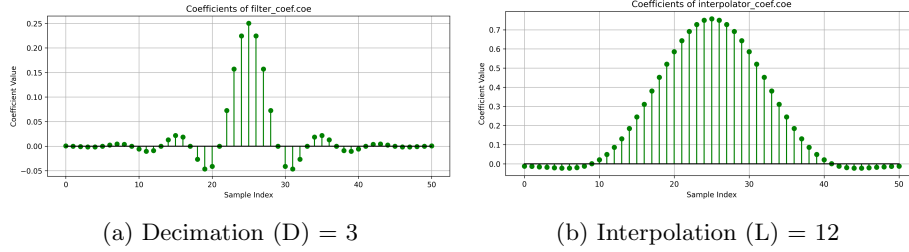


Figure 2: coefficients of filter_coef.coe and interpolator_coef.coe